

BUILD A CLASSROOM DASHBOARD THAT REMEMBERS

One file that runs your room: a class list, a fair name picker, a group maker, a timer and a points tracker — and it still knows all of it tomorrow morning. No login, no subscription, no account, nothing uploaded anywhere.

10

tasks, each one finished before the next

12

prompts, written out in full

1

finished app, included

This is the one that turns a page into software.

The difference between an activity and an application is that an application *remembers*. Task 2 is where that happens, and once you have done it once you can put it in anything you build.

Where your data actually lives

On your device, in your browser, and nowhere else. It is not uploaded, not synced, and not readable by us or anyone else. Page 5 explains exactly what that means for putting student names in — including the moment it stops being true.

What you will have at the end

Six tools in one file. Open it on the board in the morning and leave it open all day.

Tab	What it does
Class list	Type your students once. It is still there next week, next term, until you clear it.
Pick a student	Random, but with fair mode : nobody gets picked twice until everybody has had a turn.
Make groups	Random groups of any size, properly shuffled, everyone placed.
Timer	One, three, five or ten minutes, big enough to read from the back.
Points	Tap to add. Totals save as you go.
Data and reset	Copy your points with the names stripped out, for when you want AI help spotting a pattern. Plus an end-of-year erase.

The fair picker is worth the build on its own

Pure random means the same three children answer everything and two never get asked. Fair mode keeps a list of who has already been picked and only draws from the rest — then starts a fresh round when everyone has had a turn. It is four lines of logic and it changes who talks in your room.

The difference between a page and an app

Build 1 in this series made an activity. This one makes software. One idea separates them.

A page

Forgets everything the moment it closes. Fine for a practice activity a student does once. Useless for anything you want to keep using — you would type your class list in every single morning.

An app

Writes what it knows into the browser's own storage on your device. Close the tab, shut the laptop, come back Thursday: still there.

The technical name is local storage, and that is nearly all you need to know about it.

It is a small notebook the browser keeps for each file on your own machine. Your app writes to it; nothing else can read it; it never leaves the device. No server, no account, no internet.

Why this is the safe option as well as the useful one

Every other way of making something remember involves sending data somewhere — a database, an account, someone's cloud. Local storage is the one option where the question "who else can see this?" has the answer "nobody". For a tool holding a class list, that is not a small detail. It is the whole reason this design is the right one.

What it cannot do, honestly

It is tied to that browser on that device. Open the file on a different laptop and it starts empty. That is a real limit, and it is the same limit that makes it private. Page 18 covers what changing it would cost.

What you need

1. An AI chat

Written with **Claude** on screen. **ChatGPT and Gemini both work** — the prompts are plain English. Where a step depends on the tool, it says so.

2. A plain text editor

Notepad, or TextEdit set to plain text. Not Word.

3. A browser, and the same one each time

This matters more here than in build 1. Your saved data belongs to the browser you used. Build it in Chrome and it lives in Chrome. Open the same file in Edge later and it will look empty — nothing is lost, you are just looking in a different notebook.

What you do not need

A server, a host, a domain, an account, a subscription, or anyone's permission for the version built here.

Time: about an hour the first time, in ten-minute pieces if that suits you better — the tasks are built to stop cleanly. Under twenty minutes once you have done it once.

The three tiers, and the names question

Build 1 avoided student data entirely. This one cannot — a name picker needs names. So here is the honest version, which is more useful than a rule that says never.

1

Content you wrote

Questions, timers, group sizes. No student in it. **Build freely.**

2

De-identified student data

S1, S2, S3 and their scores. The dashboard's export tab produces exactly this. **Safe to paste into an AI chat** because nobody can be identified from it.

3

Data traceable to a child

Names, and anything attached to them. **Whether this is fine or a problem depends entirely on where it lives.** That is the point of this page.

The same names, two completely different situations

This file on your laptop

Saves only to your browser. Nothing transmitted. No account. No link anyone could visit. About as exposed as the class list in your planner.

The same file hosted online

Now reachable by a URL. Now a record held somewhere, by software your district did not review.
Now a conversation with your district, not a setting you flip.

So the rule is not "never use names." It is: **names may live locally on your own device; they may not be published, synced, or pasted into a chatbot.** The dashboard is built that way on purpose, and the export tab exists so you never have to break it.

Practical guidance, not legal advice. FERPA, COPPA, your state law and your district's policy are the authorities. If you are unsure, use student numbers instead of names — the dashboard works exactly the same.

Ten tasks, and how each one ends

Every task below finishes with something you can see working. If it does not work, you do not move on — you say what went wrong and ask again. That is the whole discipline.

1 Build the shell

Six tabs that switch. Nothing in them yet.

2 Make it remember

The one that matters. Close the tab, reopen, data still there.

3 The class list

Add and remove students. Survives a reload.

4 The fair picker

Random, but nobody twice until everyone has had a turn.

5 Groups

Properly shuffled, everybody placed, no leftovers dropped.

6 Timer

Readable from the back of the room.

7 Points

Tap to add; totals save as you go.

8 The de-identified export

The safety feature, and the bridge to asking AI for help.

9 Make it yours

Your colors, your wording.

10 Make it usable by everyone

Keyboard, contrast, screen readers.

Stop wherever you like. A dashboard with tasks 1 to 4 done is already better than most of what is installed on your school laptop. The rest can wait for next weekend.

TASK 1

Build the shell

PROMPT 1 • THE SHELL

Build me a classroom dashboard as one single HTML file I can save and open in a browser.

It has six tabs across the top: Class list, Pick a student, Make groups, Timer, Points, and Data and reset. Clicking a tab shows that panel and hides the others.

Requirements:

- Everything in ONE file — styling and code included. No internet, no login, no account.
- Big text and big buttons; this gets used on a board at the front of a room.
- Leave each panel empty for now with just its name in it. I am going to fill them in one at a time.
- Nothing is sent anywhere and nothing asks for a password.

Task 1 is done when

You can click all six tabs and watch the panel underneath change. That is all. Do not add anything else yet — building one piece at a time is what stops the conversation tying itself in knots later.

If it gave you several files

Say: *"Put all of that into one single HTML file so I only have one thing to save."*

Make it remember

Everything else in this build is ordinary. This is the step that makes it software.

PROMPT 2 · SAVING

Now make this dashboard remember its data between sessions, using the browser's local storage on my own device. Nothing should be sent to a server and there should be no account.

Keep everything in one saved object: the class list, the points for each student, and which students have already been picked.

Important:

- Save whenever anything changes, automatically. I should never press a Save button.
- Load whatever was saved when the page opens.
- If saving or loading fails for any reason, the app must still work with empty data rather than showing an error or a blank screen.
- Show a small "Saved" confirmation so I can see it happening.

Why that fourth bullet is in there

Local storage can be switched off. A private window blocks it, some managed school devices restrict it, and a browser set to clear site data will wipe it. Without that instruction the app can crash on open. With it, the worst case is an empty class list — annoying, not broken. **Asking for the failure case is the single most useful habit in this whole series.**

The test that proves it

Do this exactly

1. Type a name into the class list. Any name.
2. **Close the tab completely.** Not refresh — close it.
3. Open the file again.

The name should still be there. If it is, you have built an application. If it is not, task 2 is not finished and nothing after this point will work properly.

IF THE TEST FAILS

I closed the tab and reopened the file, and my data was gone. It should have been saved to local storage on this device and loaded again when the page opens. Fix that and give me the whole file back.

Do not move past this page until the test passes. Every later task saves something, and debugging saving while also debugging a points tracker is much harder than debugging saving on its own. This is the one place in the build where patience genuinely pays.

A thing worth knowing

Your data belongs to *that file in that browser*. Move the file to a different folder and it usually still works; open it in a different browser and it starts fresh. Nothing is lost — you are looking in a different notebook.

TASK 3

The class list

PROMPT 3 · THE ROSTER

In the Class list tab: a text box and an Add button. Typing a name and pressing Add, or pressing Enter, adds that student to a list shown as rounded chips underneath, each with a small x to remove it.

- Adding or removing saves straight away.
- Refuse duplicates and say so, rather than adding the same name twice.
- Removing a student also removes their points and their pick history.
- Show a running count underneath: "24 students".

That third bullet is not fussiness

When a child moves away mid-year and you delete them, their points should go too. Without that instruction their total sits in storage forever and shows up in your export months later. **Say what should happen to the data when something is deleted** — it is the kind of thing an AI will not decide sensibly on its own.

Task 3 is done when

You can add your real class, reload the page, and they are all still there. At this point the thing is genuinely useful even with four tabs still empty.

TASK 4

The fair picker

The one feature here that most bought tools get wrong.

PROMPT 4 · FAIR RANDOM

In the Pick a student tab: a big Pick button that shows one student's name in large text.

Add a checkbox called "Fair mode", on by default. When fair mode is on, remember who has already been picked and only choose from students who have not been picked yet. When everyone has had a turn, start a fresh round automatically.

Underneath, show how many students are still to be picked this round. The pick history saves with everything else.

Why this matters more than it sounds

Truly random means the same three children answer everything while two are never asked all week — and you will not notice, because each individual pick looked fair. Fair mode makes participation even by design instead of by your memory. **It is the single most defensible thing in this dashboard if a colleague asks why you built your own.**

Task 4 is done when

With three names on the list, three picks give you three different students, and the counter goes 2, then 1, then 0 — and the fourth pick starts a new round.

Groups and the timer

PROMPT 5 · GROUP MAKER

In the Make groups tab: a number box for group size (default 3) and a Make groups button. It shuffles the whole class properly and deals them into groups of that size, shown as cards side by side.

- Everybody must end up in a group. If the class does not divide evenly, the last group is smaller — never drop anyone.
- Use a proper shuffle, not sorting by a random number.

Why that last line is there

The common quick way to shuffle a list produces uneven results — some arrangements come up far more often than others. Asking for a proper shuffle costs you nothing and means your groups really are random. You do not need to know the algorithm; you just need to have asked for it.

PROMPT 6 · TIMER

In the Timer tab: a big countdown in minutes and seconds, readable from the back of a classroom. Preset buttons for 1, 3, 5 and 10 minutes, plus Start and Pause. When it reaches zero it stops and turns red. It does not need to save between sessions.

Done when

Groups: everyone from your class appears exactly once across the cards. Timer: it counts down, pauses, and turns red at zero.

Points, and the export that keeps you safe

PROMPT 7 · POINTS

In the Points tab: a table with one row per student showing their name, their total, and +1 and +5 buttons. Tapping a button adds to that student's total and saves straight away.

PROMPT 8 · THE DE-IDENTIFIED EXPORT

In the Data and reset tab: a button that builds a copy of the points list with **every name replaced by S1, S2, S3** and so on, shown in a text box, plus a Copy button.

Add a clear note on the page saying this de-identified version is the one to paste into an AI chat, and that the version with names should stay on this device.

Also add two buttons that ask for confirmation first: Clear all points, and Erase everything.

Task 8 is the one that makes the rest of it responsible.

Without it, the first time you want AI help spotting a pattern you will be tempted to paste the real list. With it, the safe option is one click and is right there next to the thing you were about to do. **Design the safe path to be the easy path** — that is a principle worth carrying into everything else you build.

Done when

The export shows S1, S2, S3 with the right totals and **no name appears anywhere in it**. Check that by eye before you trust it.

Make it yours, and make it usable by everyone

PROMPT 9 · APPEARANCE

Restyle it with [your colors] as the main colors. Make the picked name and the timer as large as will fit. Round the corners and give the panels a thick outline so it reads clearly from a distance.

PROMPT 10 · ACCESSIBILITY, AND THIS IS NOT OPTIONAL

Make sure everything works for someone using a keyboard instead of a mouse: every tab and button reachable with Tab, usable with Enter, with a clearly visible outline on whatever is selected. Do not rely on color alone to show state — include words. Give every icon-only button a label a screen reader can announce.

How to check prompt 10 actually worked

Put the mouse down. Press Tab repeatedly — a clear outline should move between the tabs and buttons. Press Enter on one — it should activate. If either fails, say exactly that and ask again. It takes a minute and it is the difference between a tool everyone in your building can use and one that quietly excludes a colleague.

One more worth adding

The small x that removes a student is an icon with no words. A screen reader announces it as "button" unless you ask for a label. Prompt 10's last line is what turns that into "Remove Ada".

Out of the chat, into your room

1. Copy all of the code

Use the copy button on the code block. Selecting by hand usually misses the last line.

2. Paste into a plain text editor

Notepad, or TextEdit set to plain text. **Not Word** — it adds invisible formatting that stops the file working, and the failure is baffling when it happens.

3. Save as dashboard.html

On Windows set "Save as type" to All Files so it does not quietly become a .txt.

4. Put it somewhere you will find it, then leave it there

This matters more than in build 1. Your saved class list is tied to this file in this browser. Pick its home now — desktop, documents, wherever — and do not move it between machines expecting the data to follow.

Make a bookmark. Open the file, then bookmark it like any web page. Now it is one click every morning instead of a hunt through folders, and it will behave like any other tool on your bar.

Using it on the board

Open it at the start of the day and leave the tab open. Press Ctrl and plus (Command and plus on a Mac) a few times to make everything bigger for the room.

The seven things that actually go wrong

1. It opens as a wall of code

The file did not save with .html on the end. Rename it. On Windows turn on file name extensions in File Explorer's View menu so you can see what it really saved as.

2. My class list vanished

Almost always a different browser, a private window, or a browser set to clear site data on exit. Check you are opening it the same way you did before. **Nothing is lost** — you are looking in a different notebook.

3. It saves, but not on the school laptop

Some managed devices block local storage. If the app still works but forgets, that is the cause — and it is why prompt 2 asked for it to keep working with empty data rather than crash. Ask your tech team; it is usually a policy setting.

4. Fair mode picks someone twice

Say: *"Fair mode picked the same student twice before everyone had a turn. It should only choose from students not yet picked this round."*

5. The group maker leaves someone out

Say: *"Three students were missing from the groups. Everyone must be placed, and the last group can be smaller."*

6. The export still shows names

Stop and fix this before using it. Say: *"The export still has real names in it. Every name must be replaced with S1, S2, S3 before it is shown."* Then check by eye again.

7. It worked, then an edit broke it

Paste the part you changed into the chat and say "something in this is broken, find it." You do not need to spot it yourself.

The same pattern, six more tools

You now know how to make something remember. That is the hard part, and it is reusable. Everything below is the same three moves: build the panel, save the data, add the export.

Build	What it stores	Tier
Seating chart	Which name sits in which seat. Print it for a substitute.	3 local
Hall pass log	Who left, when, and when they came back.	3 local
Reading log	Minutes read per student per week, and a running total.	3 local
Behavior tally	Counts of a specific behavior across a week.	3 local
Supply or equipment sign-out	Which laptop or calculator went to whom.	3 local
Class agenda page	Today's plan and links. No student data at all.	1

Notice the tier column, and notice the word *local*.

Five of those hold names, and all five are fine for the same reason this dashboard is: the data stays on your device. The tier is not decided by whether names appear. It is decided by **where the data lives and who else can reach it**. That is the sentence to remember out of this whole pack.

The prompt shape that builds any of them

"Build me a [thing] as one single HTML file. It should [what the teacher does]. Save everything to local storage on my device so it is still there tomorrow, and keep working with empty data if saving is unavailable. Add a button that copies a version with names replaced by S1, S2, S3. No account, no login, nothing sent anywhere."

What hosting would cost you

Sooner or later you will want the dashboard on your phone as well as your laptop, or a colleague will ask for a copy that stays in sync. Here is what that actually involves, so you can decide with your eyes open.

What changes the moment it goes online

- The data stops living on your device and starts living on someone's server.
- A link exists that could reach it.
- It becomes software holding student records that your district did not review.
- You, personally, become responsible for something that is properly the school's responsibility.

What to do instead of abandoning the idea

Take it to your technology team as a proposal rather than a surprise. Bring what it does, what data it touches, and where it would live. Districts say yes to reviewed things far more often than to discovered things — and a district that has seen it can host it properly, which is better than you hosting it badly.

The middle option most teachers actually want

Keep it local, and use the export. If you want the same dashboard on two machines, you do not need syncing — you need to retype a class list once a year. That is twenty minutes against a genuine compliance question, and for most people it is not a close call.

The honest summary: everything in this pack is yours, private, and needs nobody's approval. The moment you want it to be in two places at once, it stops being yours alone. That is not a rule someone invented to slow you down — it is just what happens when data moves.

Practical guidance, not legal advice. Your district's policy, your state's student privacy law, FERPA and COPPA are the authorities.

Every prompt, in one place

Copy from here rather than retyping. Square brackets mean put your own thing in.

1 • THE SHELL

Build me a classroom dashboard as one single HTML file. Six tabs across the top: Class list, Pick a student, Make groups, Timer, Points, Data and reset. Clicking a tab shows that panel and hides the others. Everything in one file, no internet, no login. Big text and big buttons for use on a board. Leave the panels empty for now.

2 • SAVING

Make it remember its data between sessions using the browser's local storage on my device. One saved object holding the class list, points, and pick history. Save automatically whenever anything changes; load on open. If saving or loading fails, keep working with empty data rather than erroring. Show a small Saved confirmation.

3 • THE ROSTER

Class list tab: a text box and Add button; Enter also adds. Names show as chips with a small x to remove. Saves straight away. Refuse duplicates and say so. Removing a student removes their points and pick history too. Show a running count.

4 • FAIR RANDOM

Pick tab: a big Pick button showing one name in large text. A Fair mode checkbox, on by default: remember who has been picked and only choose from those who have not, starting a fresh round when everyone has had a turn. Show how many are left this round. Pick history saves.

5 • GROUPS

Groups tab: a group size box and a Make groups button. Shuffle properly and deal into groups of that size, shown as cards. Everyone must be placed; the last group can be smaller. Use a proper shuffle, not sorting by a random number.

6 • TIMER

Timer tab: a big countdown readable from the back of a room. Presets for 1, 3, 5 and 10 minutes, plus Start and Pause. Stops and turns red at zero. Does not need to save.

The rest of the prompts

7 • POINTS

Points tab: a table with one row per student showing name, total, and +1 and +5 buttons. Tapping adds and saves straight away.

8 • DE-IDENTIFIED EXPORT

Data tab: a button that builds a copy of the points with every name replaced by S1, S2, S3, shown in a text box, plus a Copy button. Add a note saying this version is the one to paste into an AI chat and the named version stays on this device. Also add Clear all points and Erase everything, both asking for confirmation.

9 • APPEARANCE

Restyle with [your colors]. Make the picked name and the timer as large as will fit. Thick outlines and rounded corners so it reads from a distance.

10 • ACCESSIBILITY

Everything reachable with Tab and usable with Enter, with a clearly visible outline on the selected item. Do not use color alone to show state. Give every icon-only button a label a screen reader can announce.

11 • WHEN SOMETHING BREAKS

Here is what I did and what happened: [what you clicked]. I expected [what should have happened]. Fix it and give me the whole file again.

12 • A PRINTABLE VERSION

Add a print stylesheet so printing gives a clean class list and points table with no buttons, no tabs and no colors — something I can put in a substitute folder.

Prompt 12 is the one teachers ask for second, right after they have used the dashboard for a week: the same data on paper, for the day the laptop does not turn up.

The part that does the remembering

The complete dashboard ships with this pack as `classroom-dashboard.html` — you do not need to type any of this. This is the piece worth actually looking at, because it is the one you will reuse in everything else you build.

```
/* ---- storage: one key, this browser only ----- */
const KEY = 'classroom-dashboard-v1';
const blank = { names: [], points: {}, picked: [] };
let S = load();

function load(){
  try {
    const raw = localStorage.getItem(KEY);
    if (!raw) return { ...blank };
    const o = JSON.parse(raw);
    return { names: o.names || [], points: o.points || {}, picked: o.picked || [] };
  } catch (e) { // private window, blocked storage, corrupt value
    return { ...blank };
  }
}

function save(){
  try {
    localStorage.setItem(KEY, JSON.stringify(S));
    flash('savedMsg', 'Saved');
  } catch (e) {
    flash('savedMsg', 'Could not save on this device');
  }
}

function flash(id, msg){
  const el = document.getElementById(id);
  if (!el) return;
  el.textContent = msg;
  setTimeout(function(){ el.textContent = ''; }, 1600);
}
```

Reading it without knowing code

load tries to read what was saved; if anything at all goes wrong it hands back empty data instead of failing. save writes the whole object and shows a confirmation; if the browser refuses, it says so rather than pretending. **Both are wrapped in try and catch**, which is the instruction from prompt 2 — keep working when storage is unavailable — showing up in the actual file.

How this pack is built

What we are claiming

That these prompts, used with a current AI chat, produce a working classroom dashboard. Every prompt was run and the resulting app was tested in a real browser before publication — including adding students, closing the tab and reopening to confirm the data survived, checking fair mode does not repeat, that the group maker places everyone, and that the export contains no names.

What we are not claiming

Not a replacement for your district's systems. Not a gradebook of record. Not a substitute for reading your district's policy. And not legal advice about student data — FERPA, COPPA, your state law and your district are the authorities.

An honest limitation

Local storage is tied to one browser on one device, and a managed school laptop may restrict it. That is a real constraint, it is the same constraint that keeps the data private, and page 18 covers what changing it would cost. AI tools also change; this pack teaches the loop and ships the finished file for exactly that reason.

Sources

The tools named on page 17 as worth building next — hall passes, seating charts, group makers, behavior trackers, name pickers — are drawn from 2026 reporting on what teachers are already building for themselves with AI. Student privacy guidance follows FERPA and COPPA; the Future of Privacy Forum's *Educator's Guide to Student Privacy* is a readable starting point. Not affiliated with any AI company or platform.

Built and audited by two certified teachers. Every prompt run, every task completed in order, and the included app tested in a browser before this pack shipped.

If this was useful

Leaving feedback earns you credit

TPT gives you credit toward future purchases every time you review something you have downloaded. Go to **My Purchases**, find this pack, leave a rating and a comment. It takes a minute and it comes back to you as money off the next thing you buy.

It also tells us what to build next, which is genuinely how these decisions get made.

If something did not work

Message us through TPT before leaving a rating and we will fix it and re-upload the same day, and you keep the updated file free. A prompt that stopped behaving is something we want to know about.

What you can do now that you could not this morning:

Build software that remembers. Not a page — an application, holding real information, running on your own machine, owned by you, costing nothing, and answerable to nobody's subscription. The dashboard was the example. **The saving was the lesson.**

Built and audited by two certified teachers

Bright Scholar · AI-Ready School